

Competitive Programming HandsOn N.1

Matteo Giuntori

September 2026

1 Check Binary Search Tree

To solve this problem, we perform a depth-first search of the tree, verifying the fundamental condition of a BST for each node. It is not sufficient to check only the direct children; we must ensure that the value of a node $node_k$ respects the bounds imposed by all its ancestors, namely a lower bound (min) and an upper bound (max):

$$min < node_k < max \quad \forall k \in Tree$$

This can be implemented recursively by propagating the constraints with the following approach:

1. Start from the root ($node_0$), without any initial bounds ($min = \text{None}$, $max = \text{None}$).
2. Verify that the current node respects the received bounds. If the condition is not met, return **False**.
3. If a left subtree is present, visit it recursively, updating the maximum bound (max) with the value of $node_k$.
4. If a right subtree is present, visit it recursively, updating the minimum bound (min) with the value of $node_k$.
5. If the checks on both subtrees are successful (or if the child nodes are empty), return **True**.

With this approach, the **time complexity** in the worst case will be $O(N)$, where N is the total number of nodes in the tree, since each node is visited exactly once.

Regarding the **space complexity**, this is determined by the cost of the recursive calls allocated on the stack. The required memory is proportional to the height of the tree: in the worst case (degenerate or completely unbalanced tree) the space complexity will be $O(N)$, while in the best or average case (balanced tree) it will be limited to $O(\log N)$.

2 Maximum Path Sum

To solve this problem, we perform a depth-first search (post-order traversal) of the tree. The core idea is to propagate the value of the best path starting from a leaf from the bottom up, simultaneously updating a global maximum value (`max_sum`) whenever a complete path between two leaves is identified.

For each visited node, the logic is divided into the following cases:

- **Base case (Leaf):** If the node is a leaf, return its value directly. This triggers the propagation of the information towards the parent node.
- **Node with both children:** Calculate the value of the complete path passing through the current node (`leaf_to_leaf`) by adding the node's key to the maximum paths coming from the left and right subtrees. If this value exceeds the global `max_sum`, update it. Finally, to continue the chain towards the upper nodes, return the value of the current node added exclusively to the *best* between the left and right paths.
- **Node with only one child:** If only the left child or only the right child is present, it is mathematically impossible to complete a path between *two* leaves at this vertex. Therefore, we do not update the global maximum, but simply return the value of the current node added to the only available path (right or left).

Maintaining a shared reference to the global maximum (`max_sum`) during the traversal is essential. This ensures that we intercept and save the optimal solution even if it is located entirely within a deep subtree, avoiding the inclusion of ancestor nodes (for example, nodes with negative values) that would inevitably degrade the overall sum.

Similarly to the first exercise, the **time complexity** for calculating the maximum path is $O(N)$, where N is the total number of nodes. This is because the algorithm performs a single complete traversal of the tree (in post-order), where each node is processed exactly once using constant-time operations (additions, comparisons, and assignments).

The **space complexity** is determined by the maximum depth reached by the recursive call stack, which corresponds to the height of the tree. In the worst case of a degenerate tree (completely unbalanced, similar to a linked list), the space complexity will be $O(N)$. In the case of a balanced tree, the height will be logarithmic, limiting the memory footprint to $O(\log N)$.